



SUFFOLK
UNIVERSITY
BOSTON

Body and Sole

In-Shoe Health Tracking Device

Arnaldo Silva

Craig W.Christensen

Abstract:

The Body and Sole device is a series of sensors located in a modified shoe that monitors body metrics such as weight, steps taken, heart rate, and BMI to allow users to make informed decisions about their daily routine or workout.

Background:

Consumers over the years have been increasingly interested in monitoring their health stats to adjust their lifestyles to reach different goals. Whether it be taking the stairs instead of the escalator to reach one's steps goals or using a scale to evaluate if one can afford to have another meal, technology is now at the center of everyone's decision making in concerns to their health. With the plethora of sensors now available to measure different metrics of the body and the increasing demand for health products, it only makes sense to have a single all-encapsulating product that grants one the ability to measure these stats whenever they would like in a product that is always worn, a shoe.

Introduction:

The Body and Sole device allows its user to make healthier decisions by measuring their weight, steps taken, calories burned, heart rate, average heart rate, BMI, and serve as a safety tool that lights up for those who run at night. These stats are visible by using a smartphone app that gets continuous regular updates and one's current heart rate is displayed on a pulsing ring light for those trying to get the most out of their cardio workout. All of these sensors are sewn in using a conductive thread on to a regular running shoe. By using conductive thread and wearable microprocessors, all of the technology needed to take these measurements are now available for use at all times. With the added benefit of the device having its own onboard Bluetooth sensor, the data is then sent over to a smartphone in a format that is easily interpretable.

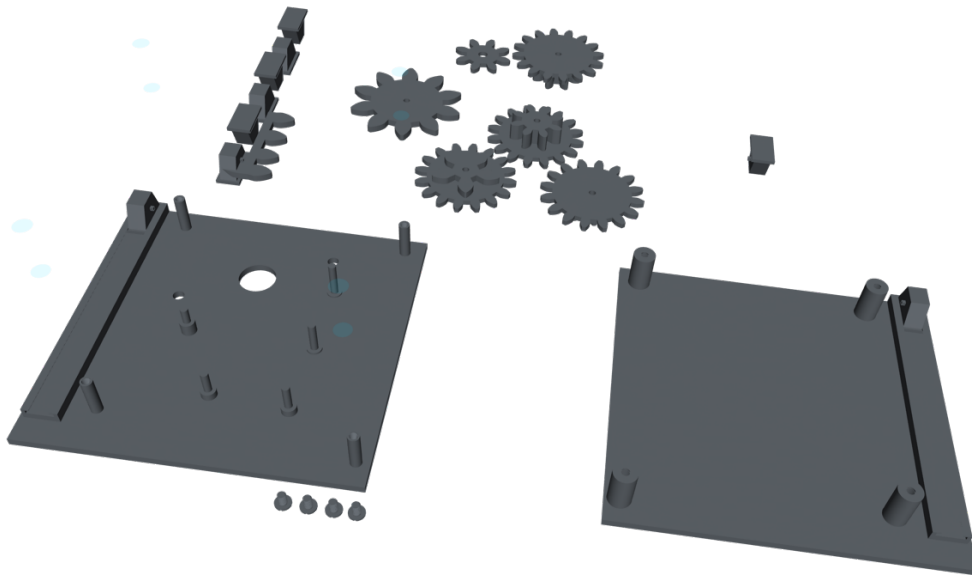
Approach:

At the center of all the sensors is a Flora, an arduino based microprocessor, that behaves as the central processing unit taking in all of the data being measured from the accelerometer, pulse sensor, and load cells through the HX711. The onboard processor on the Flora then interprets the data from the code used in the unit to then transmit it to the Bluefruit LE sensor that connects to a smartphone that receives the data in a readable format. In the initial concept of the project, it was envisioned to include a gear box system that would convert the energy in each step to electrical energy but due to 3D-Printing constraints, none of the models worked. The following demonstrates the steps taken to execute the idea to provide a working concept.

3D Modeling

After taking online classes on both mechanical engineering and 3D-modeling, the Siemens' CAD software Shapr was selected to model a theoretical working model that would be capable to converting kinetic energy into electrical energy. Figure 1 portrays the model that was created that is functional in a computational trial.

Figure 1



Fabrication

Using multiple 3D printers, the model was printed out on an Ultimaker 2+, MakerBot Replicator +, and the Flashforge Guider II but all models did not print correctly for various reason including proportion being off, pieces coming out slanted or parts just not printing out at all.

Realization

After realizing that this model would require industrial machinery, the self-recharging aspect of the project was abandoned.

Component Testing

Before attaching any of the sensors to its final placement, I tested out each individual sensor to analyze their functionality and how they could be used together in the final

product. This was done by utilizing the example sketches that are included in their respective libraries. Each sensor had their own library that needed to be downloaded in the Arduino IDE Library Manager. With the libraries installed, the example sketches utilized are shown in the Appendix.

Calibration

The first sensor to be tested was the HX711 that is attached to 50kg load cells then attached alone to the Flora using the same pins used in Figure 2. Appendix A is the code utilized to test out the weight that is being registered by the HX711. Using myself as a calibrator, I know my own weight so the number in **scale.set_scale(1000.f);** had to be changed various times until the weight registered was accurate. The number started off at 2500 but after various trials, the most accurate readings came from 1000.

Pulse Sensor Calibration and Comparison

After completing calibration, the pulse sensor was to be tested alone using the code in Appendix B to map out the heartbeat measured. By default, the threshold value was 550 so any values exceeding that limit were read as a heartbeat. Using this code, the sensor was tested while placed on different parts of my leg and foot to find the best location to place the sensor. The sensor picked up the most accurate readings when it was placed at the lower part of the shin. I wore the sensor there for about half an hour to make sure the threshold was accurate, without having found any anomalies with the values read by comparing it to the Apple Watch's readings, the threshold remained at 550.

Accelerometer Functionality Check

Next, the accelerometer was tested alone using the code in Appendix C. This code was purely just used to observe how the x,y, and z values changed with movement and to make sure the sensor was working.

Bluefruit/Bluetooth Testing

To test out the Bluefruit LE, Appendix D allowed the sensor to be connected to the Adafruit app “Bluefruit” on any smartphone and transmit data back and forth. With the code tested out when the Bluefruit LE sensor was attached alone; it was now time to combine all of the sensors and their code to complete the goal of them working cohesively as an activity tracker.

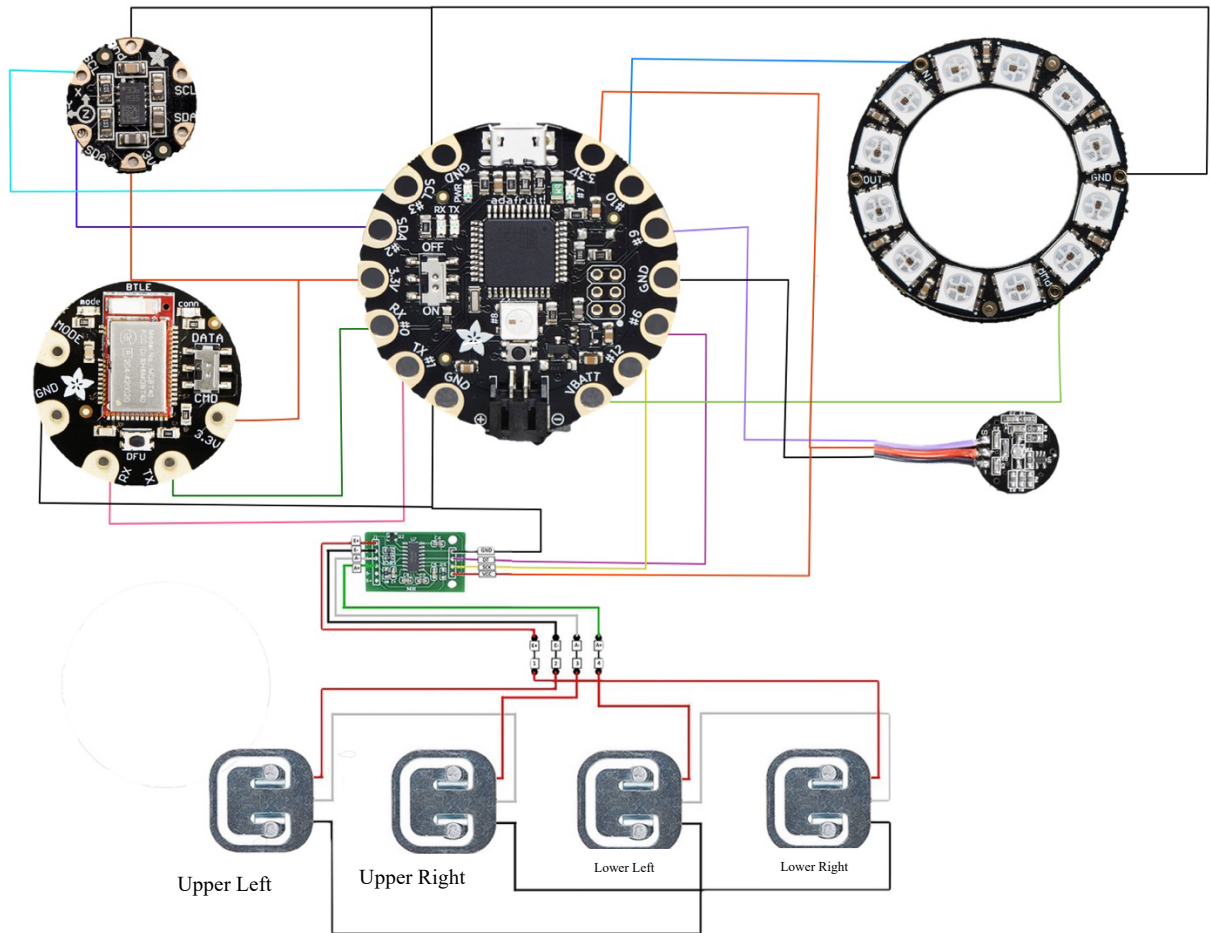
Final Code Write-Up

Using all of the example sketches provided, I consolidated their core functionality in to the code seen in Appendix E to be able to track steps taken, heart rate, average heart rate, weight, and total calories burned.

Sewing/Soldering

A conductive thread was used for its versatility as a thread without losing the functionality of a standard cable. Figure 2 demonstrates how the components were laid out to be sewn on the shoe.

Figure 2

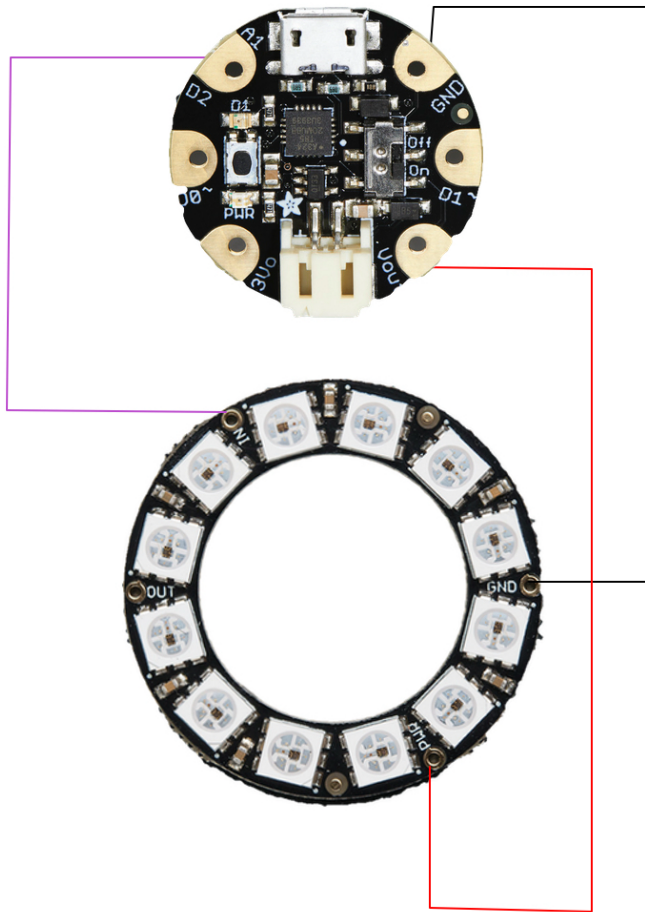


Second Shoe Sewing and Schematics

Similarly, the second shoe also had a processor on a Gemma board that was only used to control a NeoPixel Ring light to serve as a safety feature for runners to be visible at night.

In Figure 3, the schematics are laid out for the set up on the Gemma and the code on it is in Appendix F.

Figure 3



Troubleshooting

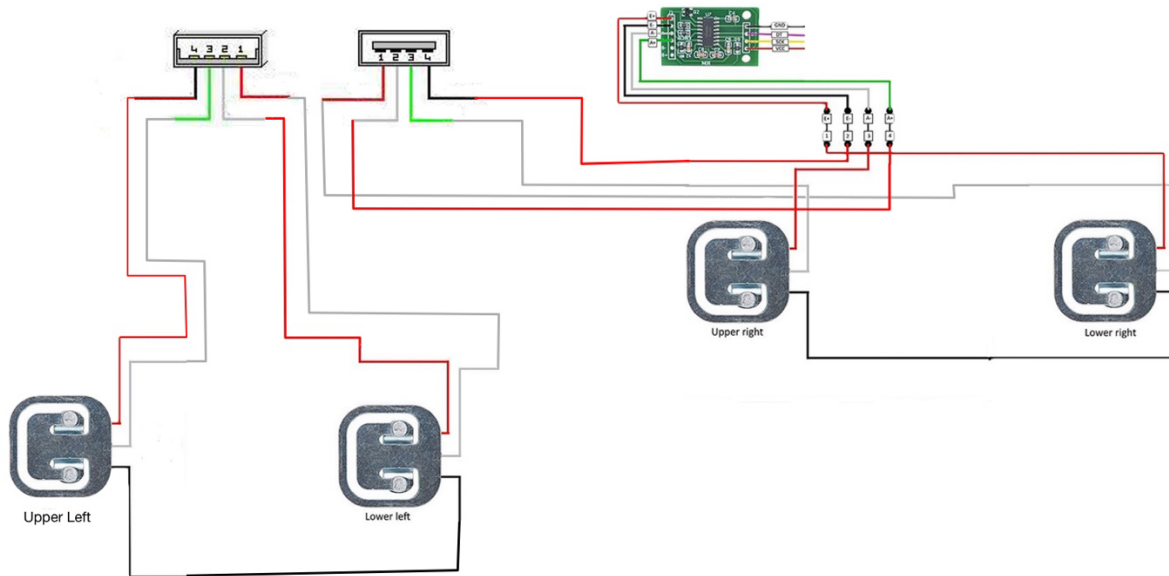
Lastly, with all components sewn and code uploaded, minor tweaks and adjustments had to be made to ensure the functionality of the device such as covering pieces with electrical tape to avoid wires from crossing each other.

Scale Separation Technique

Because the HX711 requires the load cells to be placed together with a load cell at the tops and bottoms of each foot, the shoes would need to always be attached to each other.

To combat that, one side of the shoe was attached to a USB cable and the other with its female counterpart as shown in Figure 4.

Figure 4



SUGGESTIONS

To ensure the functionality of the wiring for the project, certain precautions and steps were taken. The pulse sensor, HX711, and the load cells were not so cooperative with the conductive thread, so those pieces used traditional wires soldered to the board. When sewing the pieces, it was especially important to allow for there to be enough space between each line of conductive thread as my project needed to be unsewn and re sewn multiple times to stop interferences.

Cost:

Shoes \$0

Flora	\$17.50
Gemma	\$12.50
Flora Accelerometer (LSM303)	\$14.95
HX711 + 50 kg Load Cells Kit	\$9.75
Flora Bluefruit LE Module	\$17.50
Gears	\$0
NeoPixel Ring	X 2 \$7.50
Lithium Ion Battery	X 2 \$9.95
Conductive Thread	\$6.95
Pulse Sensor	\$25.00
Total Cost	\$139.05

Performance

To test how accurate the final project is, the shoe was worn for about ten minutes at a time and then reset and compared to the values on an Apple Watch and a scale. In the testing done, the steps taken were very accurate and very close to what the Watch was counting within about 5 steps from each other. For the calories burned, I could not really compare to the watch as it is unknown how they calculate that metric and for my project I used an equation from the following website <https://healthfully.com/93820-accurately-calculate-bmi.html> to obtain the metrics for calories burned. For heart rate, I also compared that to the Apple Watch and the majority of the time the heart rate was very close to that of the watch within about 10 BPM but anomalies appeared at times when moving excessively. For the weight, this metric was a bit of a wild card as the values changed dramatically depending on the surface one was stepping on, but I

did find that when stepping on tile or wood floor, it typically got within 15 pounds of my actual weight as compared to a scale.

Project Limitations

The biggest limitation encountered was not having the right equipment that would be able to reproduce the 3D model I had made in Shapr. Without that equipment I had no way of creating the gear box needed for the shoe to recharge with every step. Instead, the project relies on a rechargeable lithium ion battery. If I had more funds and time, I would have had the 3D model manufactured using metal as the plastic models were too flimsy and did not print out precisely enough to work.

Demo

When using the device, the user needs to download the Bluefruit app from a smartphone. After it is downloaded, one can flip the switch on the Flora to the “ON” position and slip on the shoes. Next, wait for the light ring to blink blue. With the device blinking blue, in the Bluefruit app, one needs to connect to the Bluefruit LE and select UART. When UART opens, send a message that says “1”, this tells the FLORA that the smartphone is now connected to the device. From there on, all instructions are sent from the Bluefruit LE to the smartphone and displayed in UART.

Areas of Improvement

Based on the tools needed to create this project, it has very high manufacturability, but trials could show that the device may not last very long. The shoe may encounter some functionality problems as time progresses as with wear, the conductive thread may corrode with sweat or thin out and snap. For the manufacturing of this product to be worth it, there might need to be some breakthroughs in a few fields. For one, there will need to be some kind of flexible and

durable conductive thread available on the market for it to withstand long runs and sweat. In addition to the conductive thread, the sensors and components would need to be housed in a way that would also be protected from sweat and liquids for it to be usable in all types of weather. Also, for it to be as sustainable as possible, there would need to be some kind of way of encapsulating all the components in a durable fashion and make it modular so the style of the shoe can be swapped out. This would allow people to not need to purchase multiple shoes to track their metrics and once the shoe wears out, the components can be reused. But as the project currently stands, it is not very sustainable due to the potential of people throwing out their shoes when they stop working introducing more electronic waste. If executed correctly by implementing these additional features and the recharging capabilities, then it would do some good by reducing the amount of tech needed to obtain such metrics and would not rely on outside sources for charge. The only concerns that may be raised for these shoes in terms of health could be that there is a lithium ion battery running it and if that were to get punctured or bent during a fall or trip it may cause a thermal event. A thermal event could be avoided as long as a future iteration of the product were to safely seal away the battery and protect it from any damage.

Summary

The Body and Sole project incorporated various aspects of electrical engineering and other engineering fields that allowed me as a student to understand how to manage myself and prepare for collaboration with others. Working alone on a project with no kind of tangible deliverables, forces one to be their own team taking on various aspects of what an office environment may look like. Having had the ability to work on a project envisioned on one's own gave one the ability to make their own path. Various obstacles that I had encountered during this

process forced me to make decisions where I had to leave out certain features or approach my project from a different angle. Specifically, for my project, I went from envisioning it as a product geared for fitness enthusiasts into something for the everyday person who wants to make decisions on their health based on the metrics they are given. This is a market that's already been tapped into by smartwatches but could only be improved by competition from a product like mine that would incorporate more metrics into the mix.

Appendix

A

```
#include "HX711.h"

// HX711 circuit wiring
const int LOADCELL_DOUT_PIN = 6;
const int LOADCELL_SCK_PIN = 12;

HX711 scale;

void setup() {
  Serial.begin(38400);
  Serial.println("HX711 Demo");

  Serial.println("Initializing the scale");

  // Initialize library with data output pin, clock input pin and gain factor.
  // Channel selection is made by passing the appropriate gain:
  // - With a gain factor of 64 or 128, channel A is selected
  // - With a gain factor of 32, channel B is selected
  // By omitting the gain factor parameter, the library
  // default "128" (Channel A) is used here.
  scale.begin(LOADCELL_DOUT_PIN, LOADCELL_SCK_PIN);

  Serial.println("Before setting up the scale:");
  Serial.print("read: \t\t");
  Serial.println(scale.read()); // print a raw reading from the ADC

  Serial.print("read average: \t\t");
  Serial.println(scale.read_average(20)); // print the average of 20 readings from the ADC

  Serial.print("get value: \t\t");
  Serial.println(scale.get_value(5)); // print the average of 5 readings from the ADC minus the tare weight
(not set yet)

  Serial.print("get units: \t\t");
  Serial.println(scale.get_units(5), 1); // print the average of 5 readings from the ADC minus tare weight (not
set) divided
// by the SCALE parameter (not set yet)

  scale.set_scale(1000.f); // this value is obtained by calibrating the scale with known weights; see the
README for details
  scale.tare(); // reset the scale to 0

  Serial.println("After setting up the scale:");

  Serial.print("read: \t\t");
  Serial.println(scale.read()); // print a raw reading from the ADC

  Serial.print("read average: \t\t");
  Serial.println(scale.read_average(20)); // print the average of 20 readings from the ADC
```

```

Serial.print("get value: \t\t");
Serial.println(scale.get_value(5));
Serial.print("get units: \t\t");
Serial.println(scale.get_units(5), 1);    // print the average of 5 readings from the ADC minus tare weight, divided
                                           // by the SCALE parameter set with set_scale

Serial.println("Readings:");
}
void loop() {
  Serial.print("one reading:\t");
  Serial.print(scale.get_units(), 1);
  Serial.print("\t| average:\t");
  Serial.println(scale.get_units(10), 1);
  scale.power_down();                    // put the ADC in sleep mode
  delay(5000);
  scale.power_up();
}

```

B

```

int PulseSensorPurplePin = 0;    // Pulse Sensor PURPLE WIRE connected to ANALOG PIN 0
int LED13 = 13; // The on-board Arduino LED

```

```

int Signal;           // holds the incoming raw data. Signal value can range from 0-1024
int Threshold = 550;  // Determine which Signal to "count as a beat", and which to ignore.

```

// The SetUp Function:

```

void setup() {
  pinMode(LED13,OUTPUT);    // pin that will blink to your heartbeat!
  Serial.begin(9600);       // Set's up Serial Communication at certain speed.
}
void loop() {
  Signal = analogRead(PulseSensorPurplePin); // Read the PulseSensor's value.
                                           // Assign this value to the "Signal" variable.
  Serial.println(Signal);    // Send the Signal value to Serial Plotter.
  if(Signal > Threshold){    // If the signal is above "550", then "turn-on" Arduino's on-Board LED.
    digitalWrite(LED13,HIGH);
  } else {
    digitalWrite(LED13,LOW); // Else, the signal must be below "550", so "turn-off" this LED.
  }
  delay(10);
}

```

C

```

#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_LSM303_U.h>
Adafruit_LSM303_Accel_Unified accel = Adafruit_LSM303_Accel_Unified(54321);

```

```

void displaySensorDetails(void)
{
  sensor_t sensor;
  accel.getSensor(&sensor);
  Serial.println("-----");
  Serial.print ("Sensor:   "); Serial.println(sensor.name);
  Serial.print ("Driver Ver: "); Serial.println(sensor.version);
  Serial.print ("Unique ID: "); Serial.println(sensor.sensor_id);

```

```

Serial.print ("Max Value:  "); Serial.print(sensor.max_value); Serial.println(" m/s^2");
Serial.print ("Min Value:  "); Serial.print(sensor.min_value); Serial.println(" m/s^2");
Serial.print ("Resolution: "); Serial.print(sensor.resolution); Serial.println(" m/s^2");
Serial.println("-----");
Serial.println("");
delay(500);
}

void setup(void)
{
#ifdef ESP8266
  while (!Serial); // will pause Zero, Leonardo, etc until serial console opens
#endif
  Serial.begin(9600);
  Serial.println("Accelerometer Test"); Serial.println("");

  /* Initialise the sensor */
  if(!accel.begin())
  {
    /* There was a problem detecting the ADXL345 ... check your connections */
    Serial.println("Ooops, no LSM303 detected ... Check your wiring!");
    while(1);
  }

  /* Display some basic information on this sensor */
  displaySensorDetails();
}

void loop(void)
{
  /* Get a new sensor event */
  sensors_event_t event;
  accel.getEvent(&event);

  Serial.print("X: "); Serial.print(event.acceleration.x); Serial.print(" ");
  Serial.print("Y: "); Serial.print(event.acceleration.y); Serial.print(" ");
  Serial.print("Z: "); Serial.print(event.acceleration.z); Serial.print(" ");Serial.println("m/s^2 ");
  delay(500);
}

```

D

```

#include <Arduino.h>
#include <SPI.h>
#include "Adafruit_BLE.h"
#include "Adafruit_BluefruitLE_SPI.h"
#include "Adafruit_BluefruitLE_UART.h"

```

```

#include "BluefruitConfig.h"

```

```

#ifdef SOFTWARE_SERIAL_AVAILABLE
  #include <SoftwareSerial.h>
#endif

```

```

/*=====
APPLICATION SETTINGS

```

```

FACTORYRESET_ENABLE      Perform a factory reset when running this sketch

MINIMUM_FIRMWARE_VERSION Minimum firmware version to have some new features
MODE_LED_BEHAVIOUR       LED activity, valid options are
                          "DISABLE" or "MODE" or "BLEUART" or
                          "HWUART" or "SPI" or "MANUAL"
                          -----*/
#define FACTORYRESET_ENABLE      1
#define MINIMUM_FIRMWARE_VERSION  "0.6.6"
#define MODE_LED_BEHAVIOUR      "MODE"
// Adafruit_BluefruitLE_UART ble(Serial1, BLUEFRUIT_UART_MODE_PIN);

// A small helper
void error(const __FlashStringHelper*err) {
  Serial.println(err);
  while (1);
}

void setup(void)
{
  while (!Serial); // required for Flora & Micro
  delay(500);

  Serial.begin(115200);
  Serial.println(F("Adafruit Bluefruit Command Mode Example"));
  Serial.println(F("-----"));
  Serial.print(F("Initialising the Bluefruit LE module: "));

  if ( !ble.begin(VERBOSE_MODE) )
  {
    error(F("Couldn't find Bluefruit, make sure it's in CoMmanD mode & check wiring?"));
  }
  Serial.println( F("OK!") );

  if ( FACTORYRESET_ENABLE )
  {
    /* Perform a factory reset to make sure everything is in a known state */
    Serial.println(F("Performing a factory reset: "));
    if ( ! ble.factoryReset() ){
      error(F("Couldn't factory reset"));
    }
  }

  /* Disable command echo from Bluefruit */
  ble.echo(false);

  Serial.println("Requesting Bluefruit info:");
  /* Print Bluefruit information */
  ble.info();

  Serial.println(F("Please use Adafruit Bluefruit LE app to connect in UART mode"));
  Serial.println(F("Then Enter characters to send to Bluefruit"));
  Serial.println();

  ble.verbose(false); // debug info is a little annoying after this point!

```

```

/* Wait for connection */
while (! ble.isConnected()) {
    delay(500);
}

// LED Activity command is only supported from 0.6.6
if ( ble.isVersionAtLeast(MINIMUM_FIRMWARE_VERSION) )
{
    // Change Mode LED Activity
    Serial.println(F("*****"));
    Serial.println(F("Change LED activity to " MODE_LED_BEHAVIOUR));
    ble.sendCommandCheckOK("AT+HWMODELED=" MODE_LED_BEHAVIOUR);
    Serial.println(F("*****"));
}
}
void loop(void)
{
    // Check for user input
    char inputs[BUFSIZE+1];

    if ( getUserInput(inputs, BUFSIZE) )
    {
        // Send characters to Bluefruit
        Serial.print("[Send] ");
        Serial.println(inputs);

        ble.print("AT+BLEUARTTX=");
        ble.println(inputs);

        // check response status
        if (! ble.waitForOK() ) {
            Serial.println(F("Failed to send?"));
        }
    }

    // Check for incoming characters from Bluefruit
    ble.println("AT+BLEUARTRX");
    ble.readline();
    if (strcmp(ble.buffer, "OK") == 0) {
        // no data
        return;
    }
    // Some data was found, its in the buffer
    Serial.print(F("[Recv] ")); Serial.println(ble.buffer);
    ble.waitForOK();
}

bool getUserInput(char buffer[], uint8_t maxSize)
{
    // timeout in 100 milliseconds
    TimeoutTimer timeout(100);

    memset(buffer, 0, maxSize);
    while( (!Serial.available()) && !timeout.expired() ) { delay(1); }
}

```

```

if ( timeout.expired() ) return false;

delay(2);
uint8_t count=0;
do
{
    count += Serial.readBytes(buffer+count, maxSize);
    delay(2);
} while( (count < maxSize) && (Serial.available()) );

return true;
}

```

E

//PULSE SENSOR

```

#define USE_ARDUINO_INTERRUPTS true // Set-up low-level interrupts for most accurate BPM math.
#include <PulseSensorPlayground.h> // Includes the PulseSensorPlayground Library.

```

```

// Variables
const int PulseWire = 9; // PulseSensor PURPLE WIRE connected to ANALOG PIN 9
const int LED13 = 13; // The on-board Arduino LED, close to PIN 13.
int Threshold = 550; // Determine which Signal to "count as a beat" and which to ignore.
// Use the "Getting Started Project" to fine-tune Threshold Value beyond default setting.
// Otherwise leave the default "550" value.

```

```

int age=21;
long bodyWeight=0;
int height=69;
long BMI;

```

```

PulseSensorPlayground pulseSensor; // Creates an instance of the PulseSensorPlayground object called
"pulseSensor"

```

```

//BLUETOOTH
#include <Arduino.h>
#include "Adafruit_BLE.h"
#include "Adafruit_BluefruitLE_SPI.h"
#include "Adafruit_BluefruitLE_UART.h"
#include "BluefruitConfig.h"
#define FACTORYRESET_ENABLE 0
#define MINIMUM_FIRMWARE_VERSION "0.6.6"
#define MODE_LED_BEHAVIOUR "MODE"
Adafruit_BluefruitLE_UART ble(BLUEFRUIT_HWSERIAL_NAME, BLUEFRUIT_UART_MODE_PIN);

```

```

//Scale
const int LOADCELL_DOUT_PIN = 6;
const int LOADCELL_SCK_PIN = 12;
#include "HX711.h"
HX711 scale;

```

```

// A small helper
void error(const __FlashStringHelper*err) {

```

```

/*Serial.println(err);*/
while (1);
}

//LEDs
#include <Adafruit_NeoPixel.h>

#ifdef __AVR__
#include <avr/power.h>
#endif

#define PIN      10
#define NUMPIXELS 12
Adafruit_NeoPixel pixels = Adafruit_NeoPixel(NUMPIXELS, PIN, NEO_GRB + NEO_KHZ800);
#define onboardPIN 8
Adafruit_NeoPixel onboard = Adafruit_NeoPixel(1, onboardPIN, NEO_GRB + NEO_KHZ800);

//Accelerometer
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_LSM303_U.h>
Adafruit_LSM303_Accel_Unified accel = Adafruit_LSM303_Accel_Unified(54321);
sensor_t sensor;

void setup() {
  Serial.begin(9600);
  Serial1.begin(9600);

  onboard.begin();
  onboard.show();
  onboard.setBrightness(100);
  onboard.setPixelColor(1, pixels.Color(255,0,0));
  onboard.show();

  analogReference(EXTERNAL);
  pixels.begin();
  //PULSE SENSOR SETUP
  delay(1000);
  pulseSensor.analogInput(PulseWire);
  pulseSensor.blinkOnPulse(LED13);
  pulseSensor.setThreshold(Threshold);
  if (pulseSensor.begin())
  {
    for(int i=0;i<NUMPIXELS;i++)
    {
      pixels.setPixelColor(i, pixels.Color(0,255,0));
      pixels.show();
      delay(100);
    }
  }
}

```

```

//BLUETOOTH SETUP
delay(5000);
ble.echo(false);
//WAIT FOR BLUETOOTH TO CONNECT
while (! ble.isConnected())
{
  for(int delayCounter = 0; delayCounter<100; delayCounter=delayCounter+10)
  {
    for(int i=0;i<NUMPIXELS;i++)
    {
      pixels.setPixelColor(i, pixels.Color(0,0,255));
    }
    pixels.setBrightness(delayCounter);
    pixels.show();
    delay(50);
  }
  for(int delayCounter1 = 100; delayCounter1!=0; delayCounter1=delayCounter1-10)
  {
    for(int i=0;i<NUMPIXELS;i++)
    {
      pixels.setPixelColor(i, pixels.Color(0,0,255));
    }
    pixels.setBrightness(delayCounter1);
    pixels.show();
    delay(50);
  }
  for(int i=0;i<NUMPIXELS;i++)
  {
    pixels.setPixelColor(i, pixels.Color(0,0,0));
    pixels.show();
  }
  delay(100);
}

//ACCELEROMETER SETUP
accel.begin();

}

double x = 0;
double y = 0;
double vector=0;
int steps=0;
float calories;
int counter = 0;
int averageHR=90;

void loop(void)
{
  if (counter == 0){
    //Body Weight Measurement
    ble.println("Sit down and make sure USB is plugged into left shoe");
    delay(10000);
  }
}

```

```

ble.println("Connecting Scale...");
scale.begin(LOADCELL_DOUT_PIN, LOADCELL_SCK_PIN);
scale.power_up();
ble.println("Calibrating Scale");
scale.set_scale(1100.f);
scale.tare();
delay(10000);
ble.println("You may now stand up");
delay(10000);
ble.println("Weighing...");
ble.print("Your weight: ");
for (int i=1; i<150; i++){
bodyWeight=((bodyWeight*i)+scale.get_units()/(i+1);
    delay(20);
}
ble.println(bodyWeight);
ble.println("Seperate Shoes");
delay(5000);
scale.power_down();
BMI = (bodyWeight/(height*height))*703;
counter++;
}

int i = 0;

    sensors_event_t event;
    accel.getEvent(&event);
    int heart_rate = pulseSensor.getBeatsPerMinute(); // Calls function on our pulseSensor object that returns BPM as
an "int".

                // "myBPM" hold this BPM value now.

if (pulseSensor.sawStartOfBeat()) {           // Constantly test to see if "a beat happened".

ble.println("Checking Health Stats:");
ble.print("BPM: ");           // Print the value inside of myBPM.
ble.println(heart_rate);
ble.print("Average BPM: ");           // Print the value inside of myBPM.
ble.println(averageHR);
ble.print("Total Calories Burned:");
ble.println(calories);
ble.print("Steps Taken:");
ble.println(steps);
ble.print("Current Weight:");
ble.println(bodyWeight);
ble.print("Height:");
ble.println(height);
ble.print("BMI:");
ble.println(BMI);
ble.print("Age:");
ble.println(age);

ble.println("\n \n \n \n \n \n");

    for(int delayCounter = 0; delayCounter<100; delayCounter=delayCounter+10){

        for(int i=0;i<NUMPIXELS;i++){

```

```

pixels.setPixelColor(i, pixels.Color(255,0,0));
}
pixels.setBrightness(delayCounter);
pixels.show();
delay(50);
}
for(int delayCounter1 = 100; delayCounter1!=0; delayCounter1=delayCounter1-10){

    for(int i=0;i<NUMPIXELS;i++){
pixels.setPixelColor(i, pixels.Color(255,0,0));
    }
    pixels.setBrightness(delayCounter1);
    pixels.show();
    delay(50);
    }
    for(int i=0;i<NUMPIXELS;i++){
pixels.setPixelColor(i, pixels.Color(0,0,0));
pixels.show();
    }

    for (i;i<25;i++){
        x=x+(event.acceleration.x);
        y=y+(event.acceleration.y);
        delay(5);
    }

    vector=sqrt(x*x+y*y);
    if (vector>260 || vector<230)
    {
        steps++;
        x=0;
        y=0;
        calories = calories+(((
55.0969+(0.6309*heart_rate)+(0.1988*bodyWeight*0.453592)+(0.2017*age))/4.184)/60)*2;
        vector=0;
        i=0;
    }
    else
    {
        x=0;
        y=0;
        vector=0;
        i=0;
    }
    averageHR=((counter*averageHR+heart_rate)/(counter+1));
    counter++;
}

}

F
#include <Adafruit_NeoPixel.h>
#ifdef __AVR__
    #include <avr/power.h>
#endif

#define PIN      2

```

```
#define NUMPIXELS    12
```

```
Adafruit_NeoPixel pixels = Adafruit_NeoPixel(NUMPIXELS, PIN, NEO_GRB + NEO_KHZ800);
```

```
void setup() {
```

```
    pixels.begin(); // This initializes the NeoPixel library.
}
```

```
void loop(void) {
    pixels.setBrightness(100);
```

```
    for(int i = 0; i < NUMPIXELS; i++){
        pixels.setPixelColor(i, pixels.Color(255,0,255));
        pixels.show();
        delay(50);
    }
```

```
    for(int i=0; i < NUMPIXELS; i++){
        pixels.setPixelColor(i, pixels.Color(255,255,0));
        pixels.show();
        delay(50);
    }
```

```
    for(int i=0; i < NUMPIXELS; i++){
        pixels.setPixelColor(i, pixels.Color(0,255,255));
        pixels.show();
        delay(50);
    }
```

```
    for(int i=0; i < NUMPIXELS; i++){
        pixels.setPixelColor(i, pixels.Color(255,255,255));
        pixels.show();
        delay(50);
    }
```

```
    for(int i=0; i < NUMPIXELS; i++){
        pixels.setPixelColor(i, pixels.Color(255,0,0));
        pixels.show();
        delay(50);
    }
```

```
    for(int i=0; i < NUMPIXELS; i++){
        pixels.setPixelColor(i, pixels.Color(0,255,0));
        pixels.show();
        delay(50);
    }
```

```
    for(int i=0; i < NUMPIXELS; i++){
        pixels.setPixelColor(i, pixels.Color(0,0,255));
        pixels.show();
        delay(50);
    }
```

```
    for(int j=0; j < NUMPIXELS; j++){
        pixels.setPixelColor(j, pixels.Color(0,0,0));
        pixels.show();
    }
```

```
delay(50);  
}  
}
```